IFT339 – Structures de données
Introduction

Manuel Lafond **Université de Sherbrooke**

IFT339

Aspects techniques
Prog
C++
IFT159

Fondements
Algos, analyse
IFT436
IFT503

IFT339

Aspects techniques
Prog
C++
IFT159

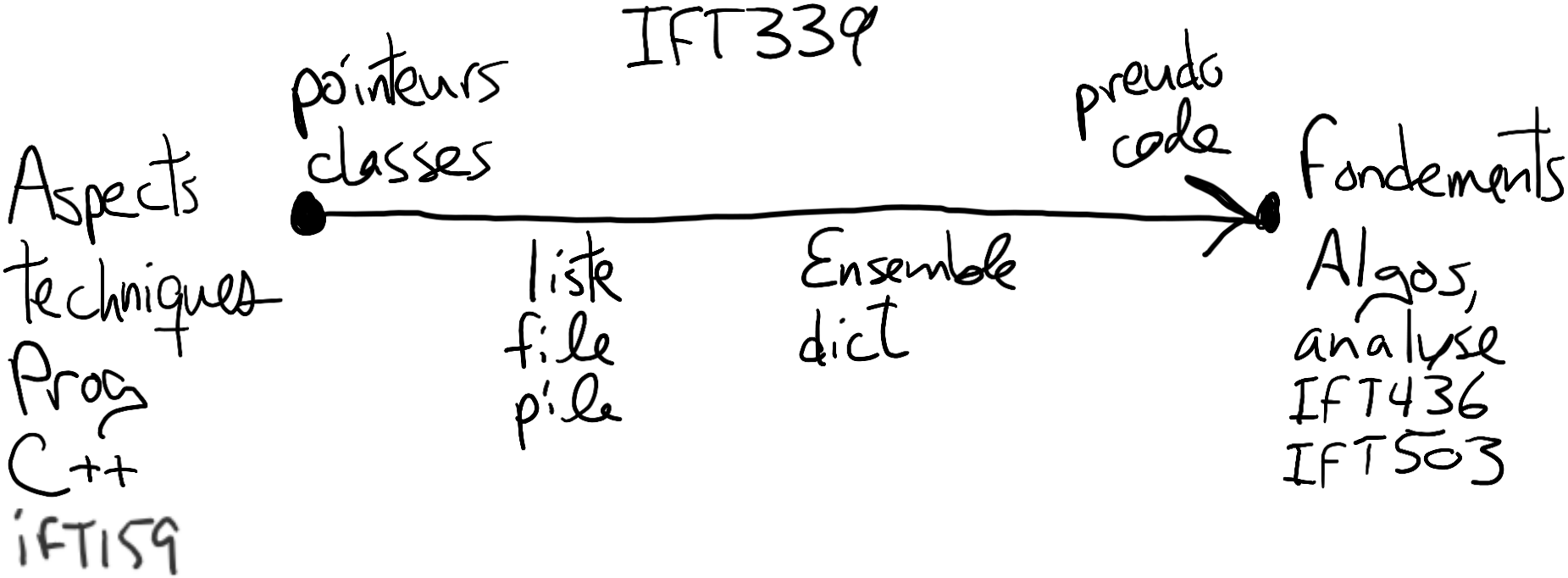
pointeurs
classes

liste
file
pile

Ensemble
dict

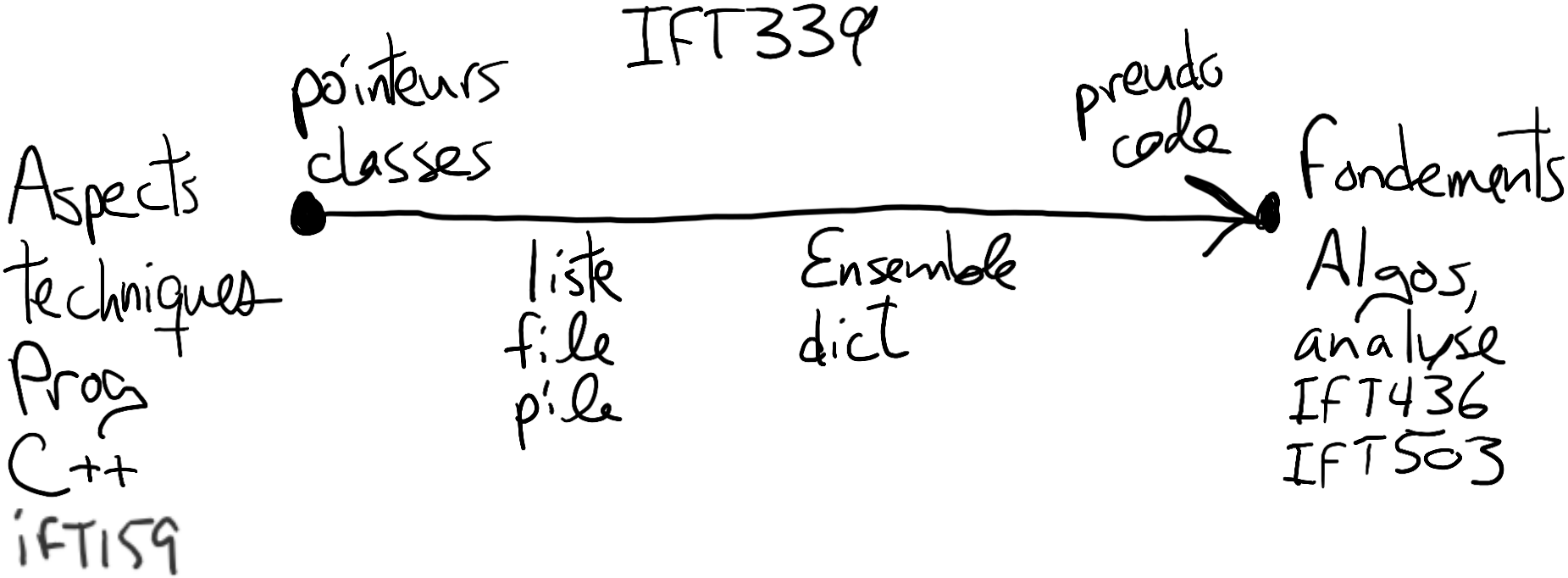
pseudo code

Fondements
Algos,
analyse
IFT436
IFT503



Objectifs

1. Comprendre l'implémentation des structures de données, leurs avantages et inconvénients.
2. Choisir la bonne structure de données pour une tâche donnée.



Objectifs

1. Comprendre l'implémentation des structures de données, leurs avantages et inconvénients.
2. Choisir la bonne structure de données pour une tâche donnée.
3. Développer un esprit analytique pour l'informatique de niveau universitaire.

Structure du cours

Partie 1. Aspects techniques

- C++, classes, gestion de la mémoire, pointeurs.
- Analyse de complexité temporelle, notation O

Partie 2. Conteneurs contigus

- Tableaux, matrices, listes chaînées, …

Partie 3. Conteneurs non-ordonnés

- Arbres de recherche, tables de hachage, graphes, …

On distingue:

Types de Données Abstraits (TDA)

vs

Implémentation (représentation)

On distingue:

Types de Données Abstraits (TDA)

vs

Implémentation (représentation)

TDA: tâches à réaliser	Impl: façon de les réaliser

On distingue:

Types de Données Abstraits (TDA)

vs

Implémentation (représentation)

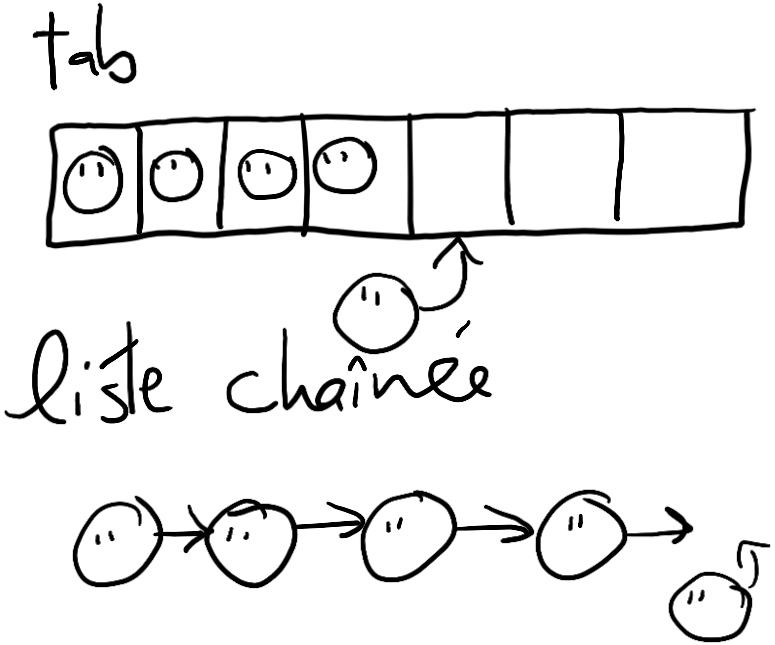
TDA: tâches à réaliser	Impl: façon de les réaliser
TDA **file** - Ajouter un élément en fin - Obtenir le premier élément - Supprimer le premier élément	

On distingue:

Types de Données Abstraits (TDA)

vs

Implémentation (représentation)

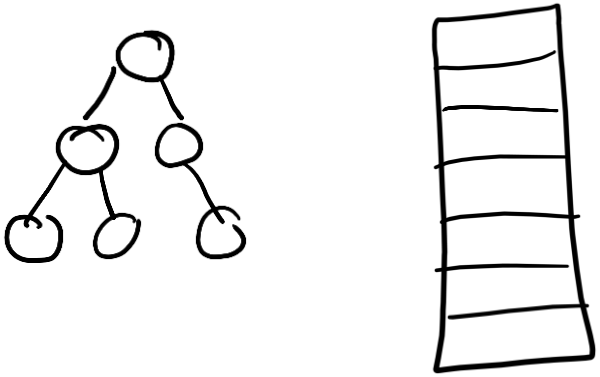
TDA: tâches à réaliser	Impl: façon de les réaliser
TDA **file** - Ajouter un élément en fin - Obtenir le premier élément - Supprimer le premier élément	

On distingue:

Types de Données Abstraits (TDA)

vs

Implémentation (représentation)

TDA: tâches à réaliser	Impl: façon de les réaliser
TDA **ensemble** - Ajouter un élément - Supprimer un élément - Savoir si un élément est présent ou non	

TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

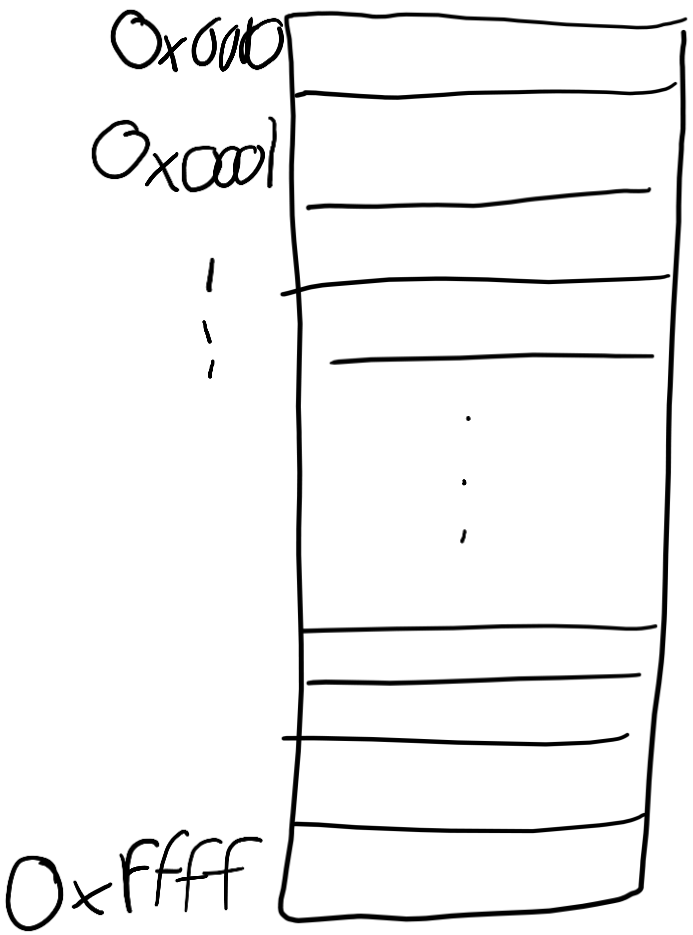
```
//alloue un espace pour x
int x = 10;
```

TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
```

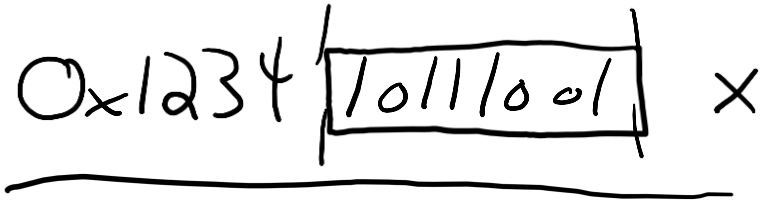


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
```

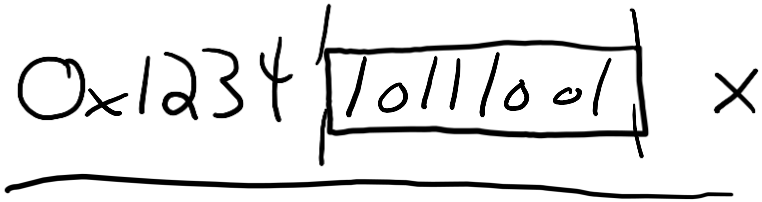


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
```

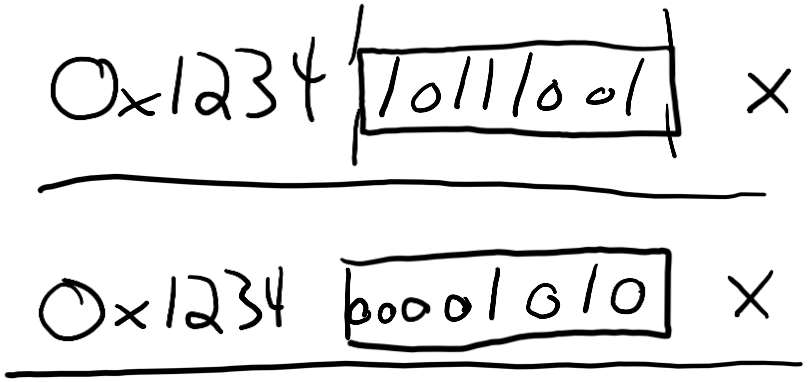


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
```

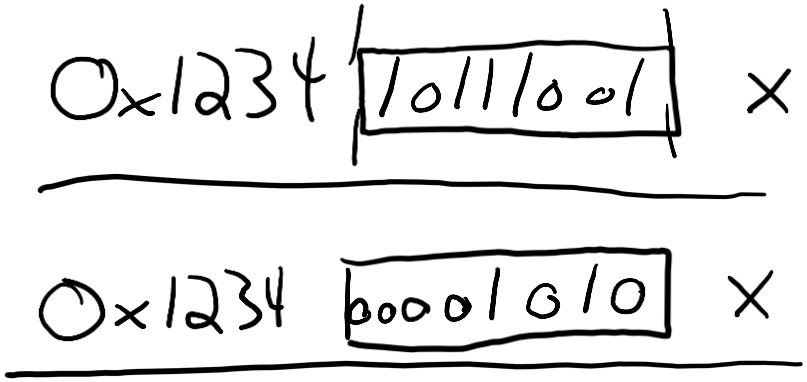


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
//alloue une mémoire non-initialisée
int y;
```

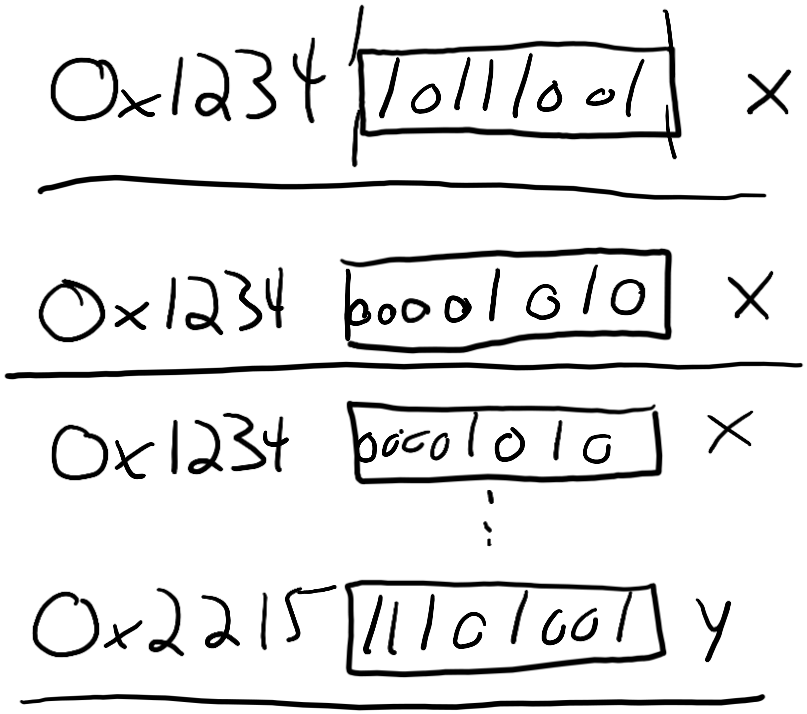


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
//alloue une mémoire non-initialisée
int y;
```

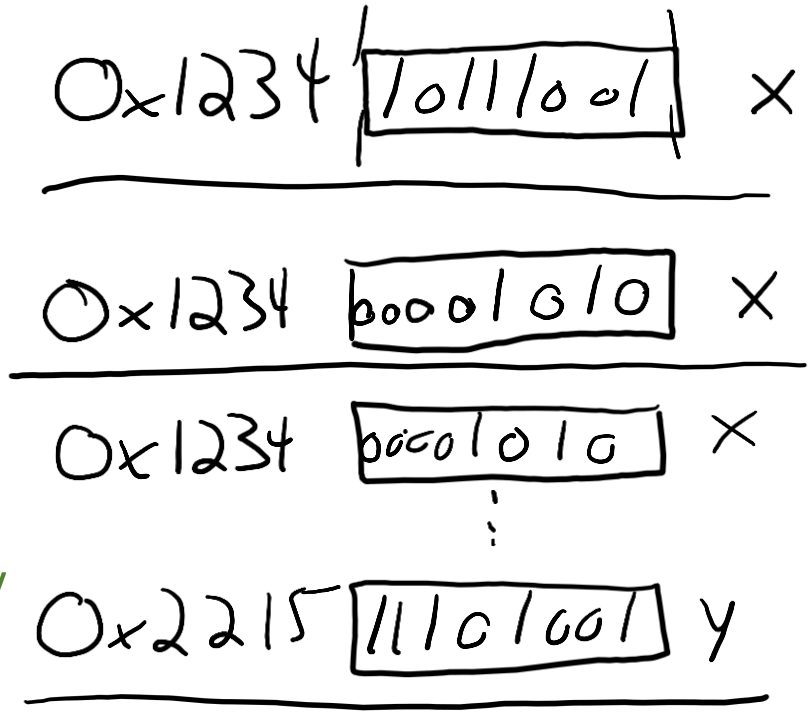


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
//alloue une mémoire non-initialisée
int y;
//COPIE le contenu de x dans la mémoire de y
y = x;
```

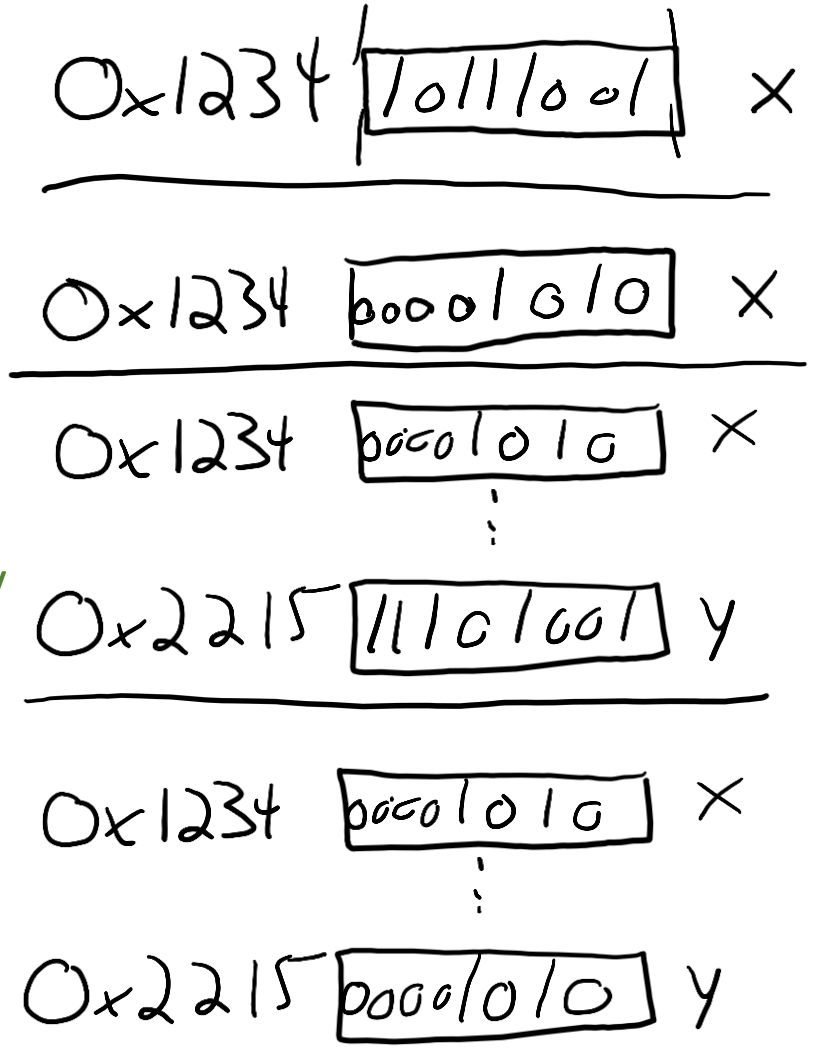


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
//alloue une mémoire non-initialisée
int y;
//COPIE le contenu de x dans la mémoire de y
y = x;
```

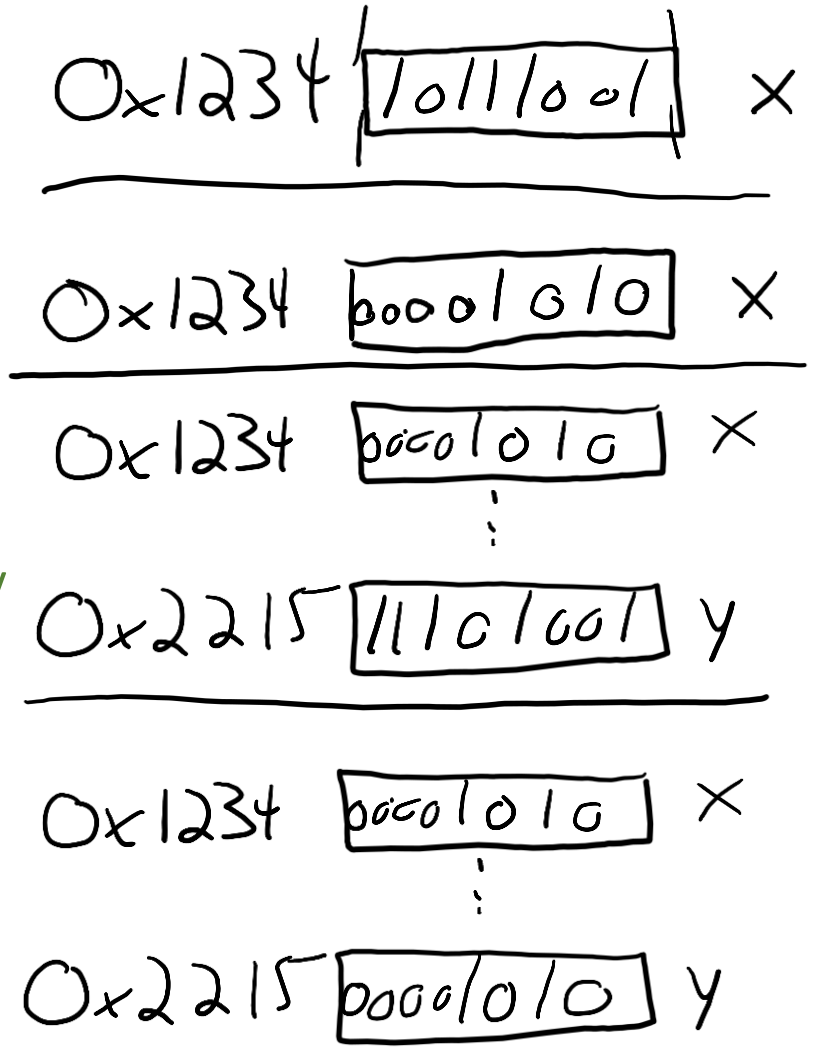


TDA: entier (représente un nombre entier)

Tâches: + - * / %

Impl C++: int, unsigned int, long, uint32_t, uint64_t, …

```
//alloue un espace pour x
int x = 10;
//copie 10 en binaire dans l'espace pour x
x = 10;
//alloue une mémoire non-initialisée
int y;
//COPIE le contenu de x dans la mémoire de y
y = x;
//y n'est pas modifié
x = 10;
```



IMPORTANT

En C++, l’assignation = fait **toujours** une copie.

a = b

copie le contenu de **b** dans la mémoire de **a**.

Même si **a** et **b** sont des objets complexes.

IMPORTANT

En C++, l'assignation = fait **toujours** une copie.

a = b

copie le contenu de **b** dans la mémoire de **a**.

Même si **a** et **b** sont des objets complexes.

```
vector<int> a, b;
for (int i = 0; i < 10000; ++i)
    b.push_back(i);
a = b;            //copie 10000 éléments
```

Passage d’arguments

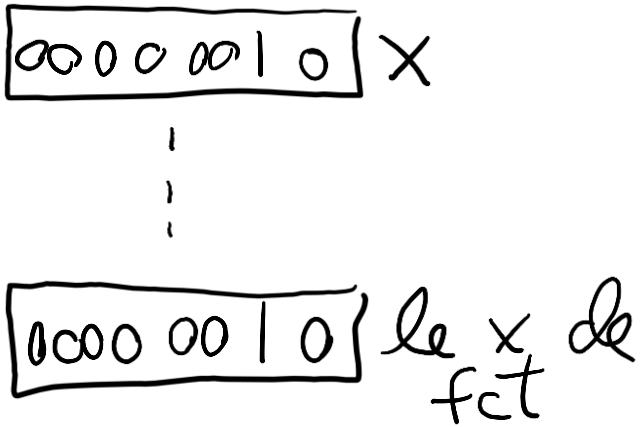
Par défaut, un argument passé à une fonction est copié.

```
void fct( int x ){
   x = 5;
}
int x = 2;
fct(x);
cout<<x;        //affiche 2
```

Passage d'arguments

Par défaut, un argument passé à une fonction est copié.

```
void fct( int x ){
   x = 5;
}
int x = 2;
fct(x);
cout<<x;          //affiche 2
```



Passage d'arguments

Par défaut, un argument passé à une fonction est copié.

Passage par valeur: une copie est faite.

Passage par référence: pour éviter la copie et passer la zone mémoire de notre variable, on ajoute & à l'argument.

```
void fct( int x ){
  x = 5;
}
int x = 2;
fct(x);
cout<<x;        //affiche 2
```

Passage d'arguments

Par défaut, un argument passé à une fonction est copié.

Passage par valeur: une copie est faite.

Passage par référence: pour éviter la copie et passer la zone mémoire de notre variable, on ajoute & à l'argument.

```
void fct( int& x ){
  x = 5;
}
int x = 2;
fct(x);
cout<<x;        //affiche 5
```

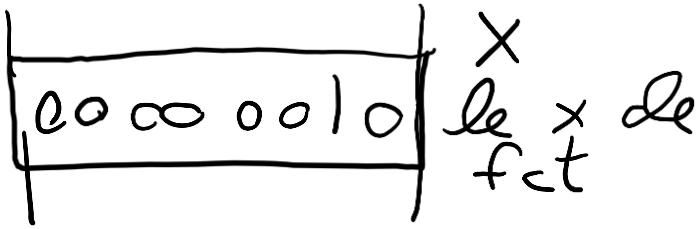
Passage d'arguments

Par défaut, un argument passé à une fonction est copié.

Passage par valeur: une copie est faite.

Passage par référence: pour éviter la copie et passer la zone mémoire de notre variable, on ajoute & à l'argument.

```
void fct( int& x ){
   x = 5;
}
int x = 2;
fct(x);
cout<<x;        //affiche 5
```



Presque toujours, un objet complexe devrait être passé par référence.

```
void fct( vector<int> v ){
   cout<<v.size()<<endl;        //gaspillage de temps et d’espace!
}
```

Presque toujours, un objet complexe devrait être passé par référence.

```
void fct( vector<int>& v ){
   cout<<v.size()<<endl;        //Bien!
}
```

Accès à l’adresse mémoire

Le &, utilisé devant une variable, retourne l’adresse de la première zone mémoire de la variable.

```
int x = 5;
cout << &x;    //affiche 010FF80C
```

Autres TDA « natifs ».

TDA **nombre réel**

C++: double, float

Autres TDA « natifs ».

TDA **nombre réel**

C++: double, float

TDA **chaîne de caractères**

C++: string

```
#include <string>
string str = "allo";
cout<<str<<endl;
```

NOTE: sizeof(montype)

Retourne le nombre d’octets réservés lorsqu’on déclare une variable de type *montype*.

Essayez:

```
cout<<sizeof(int)<<endl;
cout<<sizeof(string)<<endl;
```

TDA **tableau de taille fixe**

int tab[4];

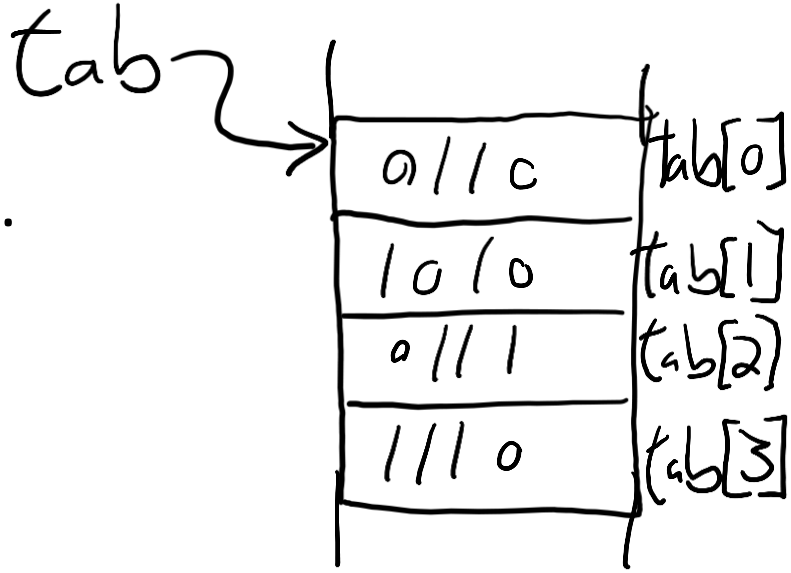
Réserve un espace contigu pour 4 int.

4 * sizeof(int)

tab est un **pointeur** vers cet espace.

Après la déclaration, l’opérateur [i] retourne le contenu mémoire de la i-ème variable à partir de ce pointeur.

TDA **tableau de taille fixe**



int tab[4];

Réserve un espace contigu pour 4 int.

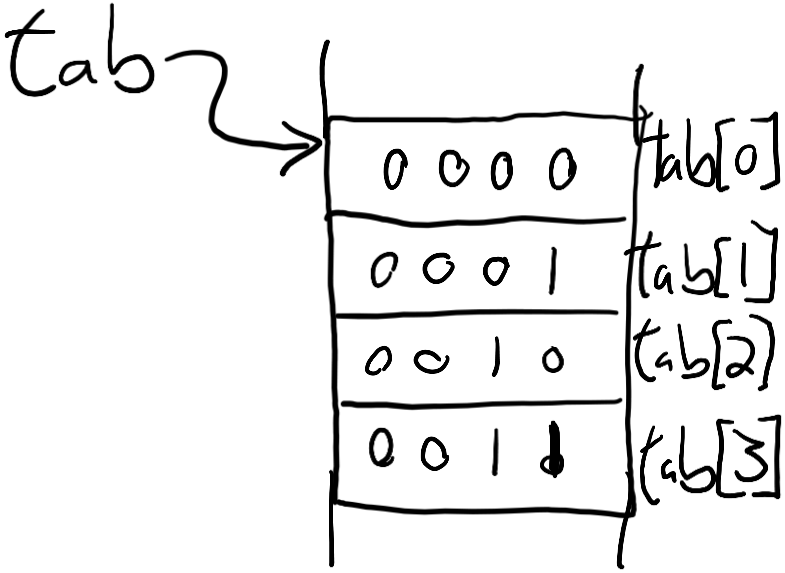
4 * sizeof(int)

tab est un **pointeur** vers cet espace.

Après la déclaration, l'opérateur [i] retourne le contenu mémoire de la i-ème variable à partir de ce pointeur.

TDA **tableau de taille fixe**

```
int tab[4];
tab[0] = 0;
tab[1] = 1;
tab[2] = 2;
tab[3] = 3;
```



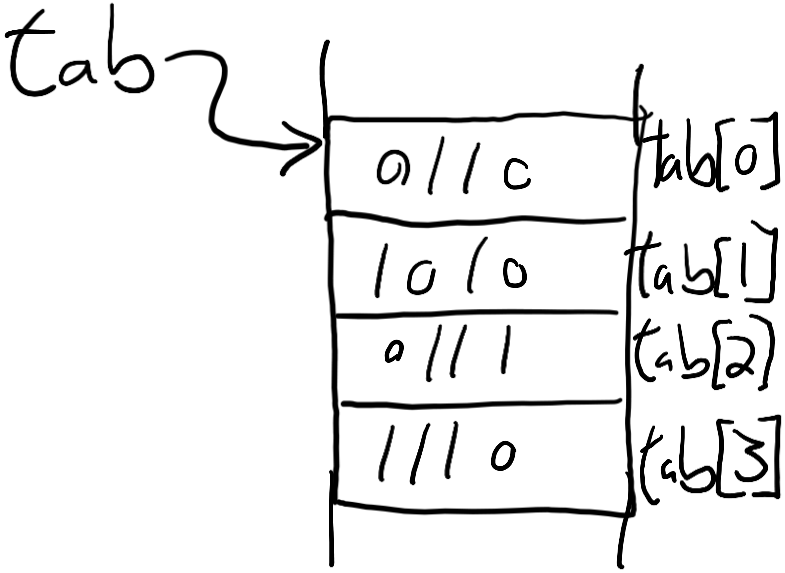

```
int x = tab[2];
x = 10;
cout << tab[2];          //affiche 2
cout << tab;             //affiche 0055FC68
```

Passage de tableaux

```
void fct(int t[4]){
        t[2] = 100;
}
int tab[4];
tab[2] = 2;
fct(tab);
cout<<tab[2];          //affiche 100.  Pourquoi?!
```

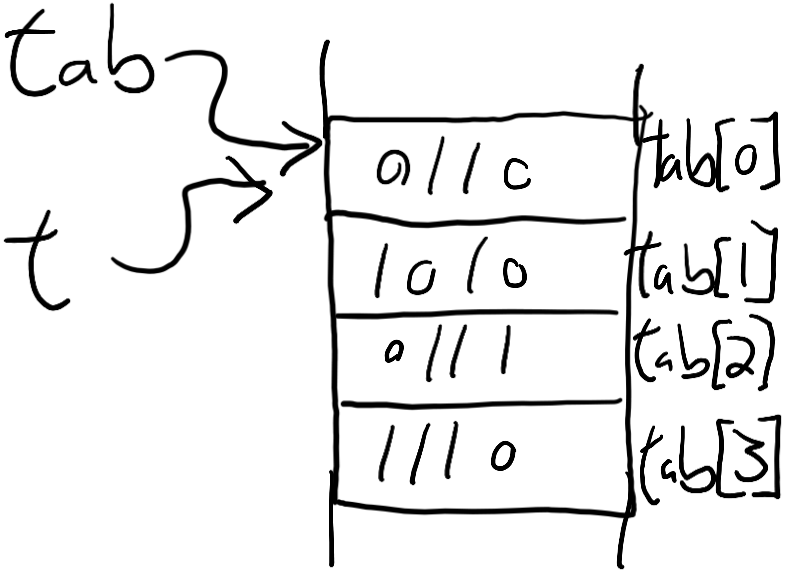
Passage de tableaux

```
void fct(int t[4]){
        t[2] = 100;
}
int tab[4];
tab[2] = 2;
fct(tab);
cout<<tab[2];          //affiche 100.  Pourquoi?!
```



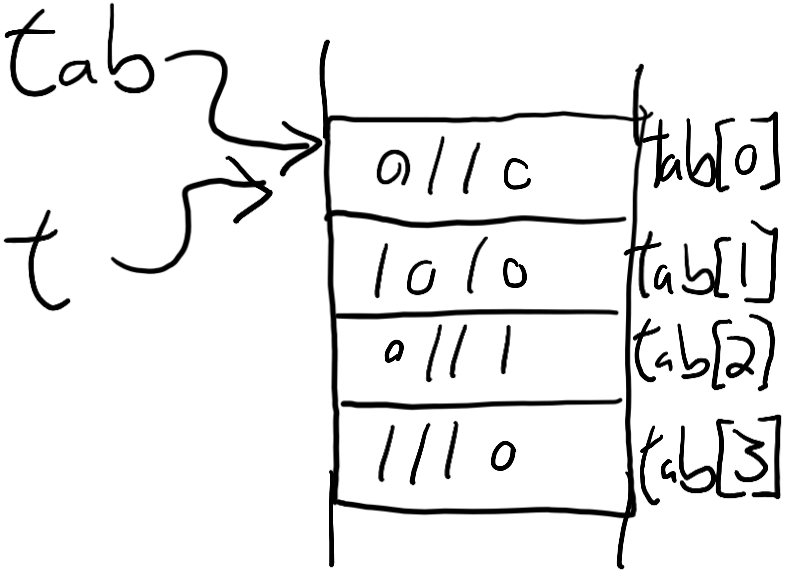
Passage de tableaux

```
void fct(int t[4]){
        t[2] = 100;
}
int tab[4];
tab[2] = 2;
fct(tab);
cout<<tab[2];          //affiche 100.  Pourquoi?!
```



Passage de tableaux

```
void fct(int t[4]){
        t[2] = 100;
}
int tab[4];
tab[2] = 2;
fct(tab);
cout<<tab[2];          //affiche 100.  Pourquoi?!
```



Au passage d'argument, le pointeur a été copié, pas le tableau.
L'opération t[2] accède à la mémoire de tab.

À connaître

- Modalités du plan de cours.
- TDA vs implémentation
- C++ aime faire des copies.
- Passage d’arguments par valeur ou par référence.
- Tableau = espace mémoire contigu
  - La variable tableau est un pointeur
  - Opérateur [ ] démarre du pointeur